

SE Lab 1 — Requirements Engineering & UML Use-Case Modelling

Problem Statement #43: Feature Flag & Dynamic Config Manager

Actors: Software Engineer, Release Manager

1. Requirements Table

Functional Requirements

ID	Description	Priority	Acceptance Criteria	Rationale
FR-001	The system shall evaluate user targeting rules (e.g., User ID hash, Beta cohort) and return accurate boolean feature flag states.	High	Pass: Target 10% rollout cohort receives enabled flag. Fail: User outside cohort receives enabled flag.	This is the core value proposition of the platform — every other feature exists to configure what this evaluation returns.
FR-002	The system shall allow a Release Manager to create and update feature flags, each with a unique key, human-readable description, and default (fallback) state.	High	Pass: A flag created with key checkout_v2 and default false is retrievable via the API immediately after creation. Fail: Duplicate keys are accepted, or the flag is not retrievable post-creation.	Flags must be individually addressable and self-documenting so multiple teams can safely manage them without key collisions.
FR-003	The system shall allow a Release Manager to configure a percentage-based rollout (0–100%) for a flag, deterministically bucketing users via a consistent hash of User ID so the same user always gets the same result.	High	Pass: Setting rollout to 25% and evaluating 10,000 distinct User IDs yields 25% ($\pm 2\%$) enabled, and re-evaluating the same IDs later returns identical results. Fail: Results vary between repeated evaluations of the same User ID at a fixed rollout %.	Gradual, deterministic rollouts are the primary risk-mitigation mechanism for shipping features safely.
FR-004	The system shall support environment-specific overrides (e.g., Development, Staging, Production) so a flag's state or rollout percentage can differ per environment.	Medium	Pass: A flag set to 100% in Staging and 0% in Production returns the correct state when the same evaluation request specifies each environment. Fail: An override in one environment leaks into another.	Teams need to test features fully in lower environments without exposing them to real users in Production.
FR-005	The system shall push real-time flag updates to all connected client SDKs (e.g., via streaming/webhook) so	High	Pass: A flag toggled by a Release Manager is reflected in a connected SDK's cache	Instant kill-switch behavior is a key selling point —

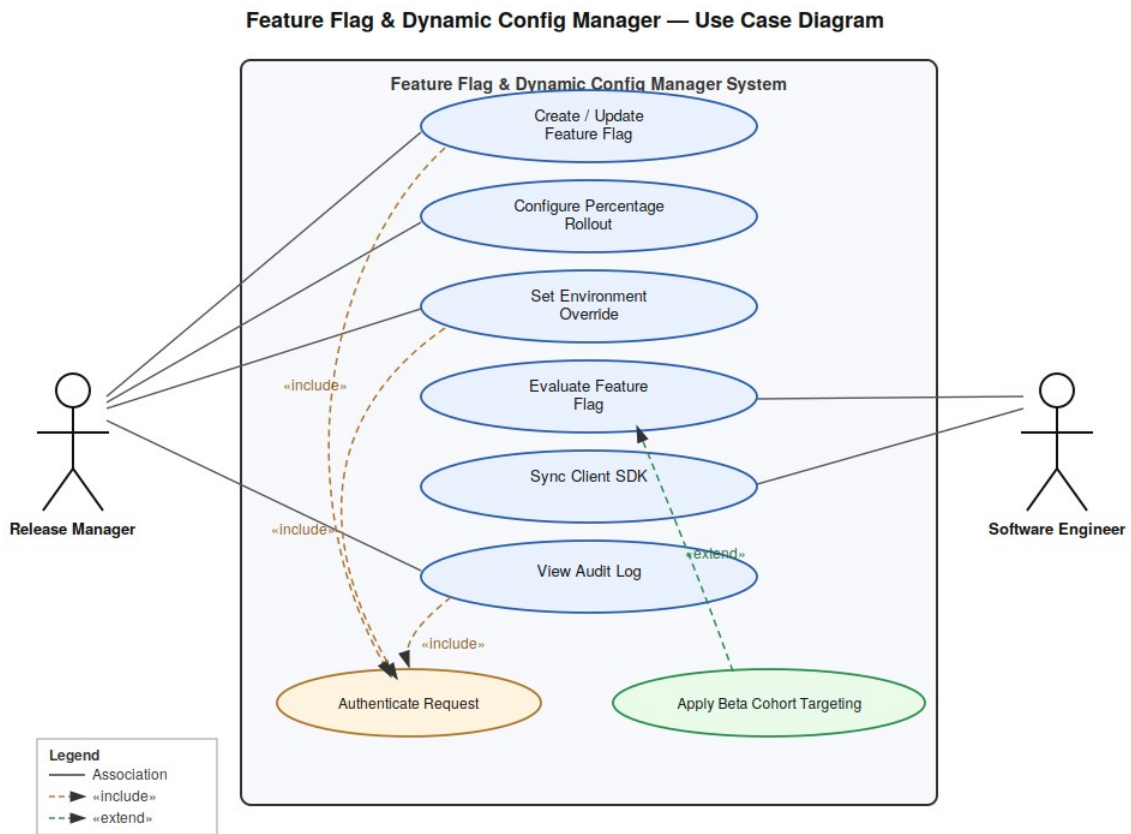
ID	Description	Priority	Acceptance Criteria	Rationale
	SDK-cached flag states are synchronized without requiring an app restart.		within the defined sync interval, without redeploying the client. Fail: SDK continues serving a stale value after the sync interval has elapsed.	waiting on a redeploy defeats the purpose of a feature flag system.

Non-Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance & Security	The feature flag evaluation API must execute in under 15 ms to minimize overhead when integrated into client critical paths, and must authenticate every request via a scoped API key.	High	Pass: Benchmarking tests confirm p99 latency under 15 ms and reject requests with missing/invalid API keys under simulated peak load.	Flag evaluation typically sits on a hot path; any added latency or security gap directly degrades the host application.
NFR-002	Reliability & Availability	The system shall maintain 99.9% uptime for the flag evaluation endpoint, and client SDKs shall gracefully degrade to the last-known-good cached flag state if the backend is unreachable.	High	Pass: During a simulated backend outage, SDKs continue serving the last synced flag values rather than erroring or defaulting to false for all flags. Fail: An outage causes the host application to crash or all flags to silently disable.	A config platform must never become a bigger single point of failure than the features it's supposed to safely control.

2. UML Use-Case Diagram

All actors, primary use cases, and both a «include» and an «extend» relationship are modeled below.



3. Use-Case Flow Specification

Use Case: Evaluate Feature Flag

Primary Actor: Software Engineer (via Client SDK)

Related Requirement: FR-001, FR-003, FR-004

Includes: Authenticate Request

Extended by: Apply Beta Cohort Targeting

Preconditions

- The feature flag being requested exists in the system and has at least one configured environment.
- The client SDK holds a valid, active API key for the target environment.
- Targeting rules (percentage rollout and/or environment overrides) have been configured for the flag by a Release Manager.

Postconditions

- A boolean flag state has been returned to the calling client SDK.
- The evaluation event (flag key, user context, environment, resulting state, timestamp) has been recorded in the audit log.

Main Success Scenario

1. The Client SDK sends a flag evaluation request to the API, supplying the flag key, environment, and user context (User ID and any relevant attributes).
2. The system authenticates the request using the supplied API key («include» Authenticate Request).
3. The system retrieves the flag's configuration, including its default state and rollout rules.
4. The system computes a deterministic hash of the User ID and compares it against the configured rollout percentage to determine cohort membership.
5. The system checks whether an environment-specific override exists for the requesting environment and applies it if present.
6. The system returns the resulting boolean flag state to the Client SDK.
7. The system logs the evaluation details to the audit trail.

Alternate Flow — A1: Environment Override Active

Trigger: At step 5, the system finds an active environment override for the requesting environment.

A1.1: The system bypasses the percentage-rollout computation from step 4 and instead applies the override value directly.

A1.2: The system returns the override value to the Client SDK.

A1.3: The system logs the evaluation, tagging it as "override-applied" in the audit trail.

The flow then rejoins the main scenario at step 7 (logging) and completes normally.

Extension — Apply Beta Cohort Targeting («extend»)

Trigger: At step 4, the user context indicates the user belongs to the registered Beta cohort, regardless of computed rollout percentage.

The system overrides the percentage-based result and returns an enabled state for that user, then rejoins the main flow at step 6.